Ubuntu Memory/Knowledge Server Architecture

Complete System Design for Shared Human & AI Brain

Executive Summary

A dedicated Ubuntu server providing unified memory and knowledge management for both humans and AI agents. This system integrates vector-based semantic search, web intelligence gathering, persistent memory layers, multi-tenant isolation, and intelligent dashboarding—creating a single source of truth for organizational knowledge.

Key Components:

- Weaviate (vector database) - semantic search and knowledge indexing
- Firecrawl (web intelligence) - intelligent document harvesting
- mem0 (memory layer) - personalized AI memory with MCP integration
- PostgreSQL (document store) - structured logging and metadata
- Custom MCP Server - bridging memory operations with AI agents
- React/Next.js Dashboard - unified control plane
- Qdrant (optional secondary vector DB) - ultra-fast similarity search for large-scale queries

---

Part 1: System Architecture Overview

1.1 Hardware Requirements

Minimum Specifications (Dedicated Server):

- CPU: 8-core (16-core recommended for multi-tenant at scale)
- RAM: 32GB (64GB for heavy concurrent indexing)
- Storage: 1TB NVMe SSD (2TB+ for document archive)
- Network: 1Gbps uplink (10Gbps for high-volume crawling)
- Dedicated machine (not shared hosting)

Recommended: Vultr/Contabo Bare Metal

- Ubuntu 24.04 LTS Server
- AMD EPYC 7002 series (excellent Docker/container performance)
- 64GB ECC RAM
- RAID-1 NVMe drives
- Per-hour or monthly billing

1.2 Core Technology Stack

Component	Purpose	Why Chosen
Weaviate	Vector search and semantic indexing	Open-source, multi-tenancy support, built-in schema management, HNSW/BQ indexing
Qdrant	Secondary vector DB (optional)	Ultra-fast retrieval, better for 100M+ vectors, gRPC protocol
Firecrawl	Web scraping and document harvesting	AI-native extraction, markdown/JSON output, self-hostable, handles JS rendering
mem0	Personalized memory layer	41K+ GitHub stars, MCP integration, conflict resolution, multi-level memory (user/session/agent)
PostgreSQL	Document/logging backbone	JSONB support, full-text search, excellent query performance
Redis	Caching and task queue	Firecrawl dependency, memory management, rate limiting
n8n	Workflow automation (optional)	Bulk processing orchestration, API integrations, scheduled crawls
MCP Server (Custom)	AI agent bridge	Custom implementation to expose memory ops as callable tools
Next.js + React	Control dashboard	Real-time system monitoring, bulk operations, tenant management

1.3 Multi-Tenant Architecture

Shared Infrastructure Model:

```
┌──────────────────────────────────────────────┐
│ Ubuntu Host (Dedicated) │
├──────────────────────────────────────────────┤
│ Docker Engine (single orchestrator) │
├──────────────────────────────────────────────┤
│ Weaviate (Multi-Tenant Schema) │
```

```
│ ├─ Global Shards (Shared) │
│ │ └─ Default vectorizer │
│ ├─ Tenant 1 Shards (Isolated) │
│ ├─ Tenant 2 Shards (Isolated) │
│ └─ Tenant N Shards (Isolated) │
├──────────────────────────────────────────────────────────────┤
│ PostgreSQL (Separate DBs per tenant) │
│ ├─ Database: tenant_1 │
│ ├─ Database: tenant_2 │
│ └─ Database: tenant_n │
├──────────────────────────────────────────────────────────────┤
│ mem0 (Tenant-isolated memory) │
│ ├─ User memories (tenant-scoped) │
│ ├─ Agent memories (tenant-scoped) │
│ └─ Session memories (tenant-scoped) │
├──────────────────────────────────────────────────────────────┤
│ Firecrawl Instance (Shared infra) │
│ └─ Per-tenant rate limiting/quotas │
├──────────────────────────────────────────────────────────────┤
│ MCP Servers (Custom, multi-tenant) │
│ ├─ Memory operations server │
│ └─ Document operations server │
├──────────────────────────────────────────────────────────────┤
│ Control Dashboard (React/Next.js) │
│ └─ Multi-tenant UI with RBAC │
└──────────────────────────────────────────────────────────────┘
```

Key Multi-Tenancy Features:

- **Weaviate**: Native multi-tenancy via tenant headers in requests
- **PostgreSQL**: Separate schemas/databases for data isolation
- **mem0**: Scoped memory objects (user_id, tenant_id, agent_id)
- **Rate Limiting**: Per-tenant quotas on Firecrawl, API calls
- **Shared Global Shards**: Optional public/shared knowledge accessible to all tenants
- **Dashboard RBAC**: Tenant isolation at UI level with JWT tokens

---

Part 2: Component Deep-Dive Installation

2.1 Host System Preparation

Install Ubuntu 24.04 LTS:

Update system

sudo apt update && sudo apt upgrade -y

Essential tools

sudo apt install -y
curl wget git build-essential
htop iotop nethogs tmux
unzip jq yq
ca-certificates apt-transport-https gnupg lsb-release

Disable unnecessary services

sudo systemctl disable bluetooth avahi-daemon

User Setup (Non-root Docker):

Create service user

sudo useradd -m -s /bin/bash memoryserver
sudo usermod -aG docker memoryserver
sudo usermod -aG sudo memoryserver

Home directory for services

sudo mkdir -p /data/{volumes,backups,configs}
sudo chown -R memoryserver:memoryserver /data

2.2 Docker & Docker Compose Setup

Install Docker Engine

curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

Verify

docker --version && docker compose version

2.3 Weaviate Vector Database Setup

Docker Compose Configuration:

/data/configs/weaviate-compose.yml

```
version: '3.8'

services:
weaviate:
image: semitechnologies/weaviate:latest
container_name: weaviate
ports:
- "8080:8080"
- "50051:50051" # gRPC endpoint
environment:
QUERY_DEFAULTS_LIMIT: 100
AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED: 'false'
AUTHENTICATION_APIKEY_ENABLED: 'true'
AUTHENTICATION_APIKEY_ALLOWED_KEYS: 'weaviate-key-prod,weaviate-key-dev'
PERSISTENCE_DATA_PATH: '/var/lib/weaviate'
DEFAULT_VECTORIZER_MODULE: 'text2vec-openai'
ENABLE_MODULES: 'text2vec-openai,generative-openai'
OPENAI_APIKEY: '${OPENAI_API_KEY}'
ENABLE_API_BASED_MODULES: 'true'
CLUSTER_HOSTNAME: 'weaviate'
MULTI_TENANCY_ENABLED: 'true'
REPLICATION_FACTOR: 1
volumes:
- weaviate_data:/var/lib/weaviate
- /data/configs/weaviate.conf.json:/etc/weaviate/weaviate.conf.json
restart: unless-stopped
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:8080/v1/.well-known/ready"]
interval: 30s
timeout: 10s
retries: 5

volumes:
weaviate_data:
driver: local
driver_opts:
type: none
o: bind
device: /data/volumes/weaviate
```

Launch Weaviate:

```
mkdir -p /data/volumes/weaviate
export OPENAI_API_KEY="sk-..." # Your OpenAI key for embeddings
cd /data/configs
docker compose -f weaviate-compose.yml up -d
```

Verify

curl -H "Authorization: Bearer weaviate-key-prod"
http://localhost:8080/v1/.well-known/ready

Weaviate Client Library (Python):

/data/scripts/weaviate_client.py

```
import weaviate
from weaviate.classes.init import AdditionalConfig, Timeout
from weaviate.classes.config import Property, DataType

client = weaviate.connect_to_local(
host="localhost",
port=8080,
grpc_port=50051,
auth_client_secret=weaviate.auth.AuthApiKey("weaviate-key-prod"),
additional_config=AdditionalConfig(
timeout=Timeout(init=30, query=60, insert=120)
)
)

def create_knowledge_collection():
"""Create multi-tenant aware collections"""
client.collections.create(
name="Documents",
description="Knowledge documents with tenant isolation",
properties=[
Property(name="title", data_type=DataType.TEXT),
Property(name="content", data_type=DataType.TEXT),
Property(name="source_url", data_type=DataType.TEXT),
Property(name="tenant_id", data_type=DataType.TEXT),
Property(name="doc_type", data_type=DataType.TEXT),
Property(name="created_at", data_type=DataType.DATE),
Property(name="metadata", data_type=DataType.OBJECT),
],
vectorizer_config=weaviate.classes.config.Configure.Vectorizer.text2vec_openai(),
multi_tenancy_config=weaviate.classes.config.Tenant(name="default_tenant")
)
print("✓ Documents collection created")
```

Execute on startup

```
if client.is_ready():
create_knowledge_collection()
```

2.4 PostgreSQL for Document Storage & Logging

Docker Setup:

/data/configs/postgres-compose.yml

```
version: '3.8'

services:
postgres:
image: postgres:16-alpine
container_name: postgres_memory
environment:
POSTGRES_USER: memoryserver
POSTGRES_PASSWORD: ${DB_PASSWORD}
POSTGRES_INITDB_ARGS: "--encoding=UTF8 --locale=en_US.UTF-8"
ports:
- "5432:5432"
volumes:
- postgres_data:/var/lib/postgresql/data
- /data/configs/init.sql:/docker-entrypoint-initdb.d/init.sql
restart: unless-stopped
healthcheck:
test: ["CMD-SHELL", "pg_isready -U memoryserver"]
interval: 10s
timeout: 5s
retries: 5

volumes:
postgres_data:
driver: local
driver_opts:
device: /data/volumes/postgres
type: none
o: bind
```

Initialize Schema (/data/configs/init.sql):

```
-- Multi-tenant document store
CREATE SCHEMA IF NOT EXISTS core;
SET search_path TO core;

-- Tenants table
CREATE TABLE tenants (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(255) NOT NULL UNIQUE,
tier VARCHAR(50) DEFAULT 'free',
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);
```

```
-- Documents with tenant isolation
CREATE TABLE documents (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
title VARCHAR(500) NOT NULL,
content TEXT,
source TEXT,
doc_type VARCHAR(50),
vectorized BOOLEAN DEFAULT FALSE,
metadata JSONB,
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_documents_tenant ON documents(tenant_id);
CREATE INDEX idx_documents_type ON documents(doc_type);
CREATE INDEX idx_documents_metadata ON documents USING GIN(metadata);

-- Crawl jobs (Firecrawl integration)
CREATE TABLE crawl_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
url TEXT NOT NULL,
status VARCHAR(50) DEFAULT 'pending',
result JSONB,
started_at TIMESTAMP,
completed_at TIMESTAMP,
created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_crawl_jobs_tenant ON crawl_jobs(tenant_id);
CREATE INDEX idx_crawl_jobs_status ON crawl_jobs(status);

-- Memory logs (mem0 integration)
CREATE TABLE memory_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
user_id VARCHAR(255),
agent_id VARCHAR(255),
session_id VARCHAR(255),
memory_type VARCHAR(50), -- 'user', 'session', 'agent'
operation VARCHAR(50), -- 'create', 'update', 'delete', 'search'
memory_content JSONB,
relevance_score FLOAT,
created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_memory_logs_tenant ON memory_logs(tenant_id);
CREATE INDEX idx_memory_logs_user ON memory_logs(user_id);
CREATE INDEX idx_memory_logs_agent ON memory_logs(agent_id);
CREATE INDEX idx_memory_logs_type ON memory_logs(memory_type);
```

```
-- Activity audit log
CREATE TABLE audit_log (
id BIGSERIAL PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
action VARCHAR(100),
actor VARCHAR(255),
details JSONB,
timestamp TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_audit_tenant ON audit_log(tenant_id);
CREATE INDEX idx_audit_timestamp ON audit_log(timestamp);
```

2.5 Firecrawl Web Intelligence Engine

Self-Hosted Firecrawl Setup:

/data/configs/firecrawl-compose.yml

```
version: '3.8'

services:
redis:
image: redis:7-alpine
container_name: firecrawl_redis
ports:
- "6379:6379"
volumes:
- redis_data:/data
restart: unless-stopped

firecrawl:
image: node:18-alpine
container_name: firecrawl
working_dir: /app
ports:
- "3002:3002"
environment:
NODE_ENV: production
REDIS_URL: redis://redis:6379
PORT: 3002
BROWSERLESS_API_URL: http://browserless:3000
MAX_CONCURRENT_SCRAPERS: 10
MAX_PAGES_PER_CRAWL: 100
SCRAPE_TIMEOUT: 60000
NAVIGATION_TIMEOUT: 30000
REQUEST_TIMEOUT: 30000
volumes:
- /data/volumes/firecrawl:/app/data
depends_on:
```

- redis
restart: unless-stopped

browserless:
image: browserless/chrome:latest
container_name: browserless
ports:
- "3000:3000"
environment:
MAX_CONCURRENT_SESSIONS: 10
TIMEOUT: 30000
restart: unless-stopped

volumes:
redis_data:
driver: local
driver_opts:
device: /data/volumes/redis
type: none
o: bind

Self-host setup reference (full deployment):

Clone and deploy

cd /data/sources
git clone https://github.com/mendableai/firecrawl.git
cd firecrawl

Deploy with Docker

docker compose up -d

Verify API

curl -X POST http://localhost:3002/v0/scrape
-H "Authorization: Bearer test-key"
-H "Content-Type: application/json"
-d '{"url": "https://example.com", "formats": ["markdown"]}'

Python Client for Firecrawl:

/data/scripts/firecrawl_client.py

import requests
import json
from typing import List, Dict

class FirecrawlClient:
def **init**(self, api_url: str = "http://localhost:3002", api_key: str = "test-key"):
self.api_url = api_url
self.api_key = api_key
self.headers = {
"Authorization": f"Bearer {api_key}",
"Content-Type": "application/json"
}

```
def scrape_page(self, url: str, formats: List[str] = ["markdown"]) -> Dict:
    """Scrape single page and return structured content"""
    response = requests.post(
        f"{self.api_url}/v0/scrape",
        headers=self.headers,
        json={
            "url": url,
            "formats": formats,
            "onlyMainContent": True,
            "timeout": 60000
        }
    )
    return response.json()

def crawl_website(self, url: str, max_pages: int = 50) -> Dict:
    """Crawl entire website"""
    response = requests.post(
        f"{self.api_url}/v0/crawl",
        headers=self.headers,
        json={
            "url": url,
            "limit": max_pages,
            "scrapeOptions": {
                "formats": ["markdown"],
                "onlyMainContent": True
            }
        }
    )
    return response.json()

def get_crawl_status(self, job_id: str) -> Dict:
```

```
    """Check crawl job status"""
    response = requests.get(
        f"{self.api_url}/v0/crawl/{job_id}",
        headers=self.headers
    )
    return response.json()
```

2.6 mem0 Persistent Memory Layer (On-Premise)

mem0 Docker Setup:

/data/configs/Dockerfile.mem0

FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y
postgresql-client redis-tools
&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "mem0_server:app", "--host", "0.0.0.0", "--port", "8000"]

mem0 Configuration (/data/configs/mem0_config.py):
from mem0 import Memory
from mem0.configs.base import MemoryConfig

Multi-tenant mem0 configuration

mem0_config = {
"vector_store": {
"provider": "weaviate",
"config": {
"collection_name": "MemoriesGlobal",
"cluster_url": "http://weaviate:8080",
"auth_client_secret": "weaviate-key-prod"
}
},
"llm": {
"provider": "openai",
"config": {
"model": "gpt-4",

```
"api_key": "${OPENAI_API_KEY}"
}
},
"embedder": {
"provider": "openai",
"config": {
"model": "text-embedding-3-small"
}
},
"version": "v1.0"
}
```

class MultiTenantMemory:
"""Scoped memory management per tenant"""

```
    def __init__(self, tenant_id: str):
        self.tenant_id = tenant_id
        self.memory = Memory.from_config(mem0_config)

    def add_memory(self, user_id: str, memory_content: str, metadata: dict = None):
        """Add memory scoped to tenant + user"""
        return self.memory.add(
            messages=[{"role": "user", "content": memory_content}],
            user_id=f"{self.tenant_id}:{user_id}",
            metadata={
                "tenant_id": self.tenant_id,
                "user_id": user_id,
                **(metadata or {})
            }
        )

    def search_memory(self, user_id: str, query: str, limit: int = 5):
        """Search tenant-scoped memories"""
        return self.memory.search(
            query=query,
            user_id=f"{self.tenant_id}:{user_id}",
            limit=limit
        )

    def update_memory(self, memory_id: str, new_content: str):
        """Update existing memory"""
```

```
        return self.memory.update(
            memory_id=memory_id,
            data=new_content
        )

    def delete_memory(self, memory_id: str):
        """Delete memory"""
        return self.memory.delete(memory_id=memory_id)
```

2.7 Custom MCP Server for Memory Operations

MCP Server Implementation (/data/scripts/memory_mcp_server.py):

```
from mcp.server import Server, stdio_server
from pydantic import BaseModel
import asyncio
import json

# Memory operation definitions

MEMORY_TOOLS = [
{
"name": "add_memory",
"description": "Add new memory to the knowledge base",
"inputSchema": {
"type": "object",
"properties": {
"user_id": {"type": "string"},
"content": {"type": "string"},
"memory_type": {"enum": ["user", "session", "agent"]},
"metadata": {"type": "object"}
},
"required": ["user_id", "content", "memory_type"]
}
},
{
"name": "search_memory",
"description": "Search memory database for relevant content",
"inputSchema": {
"type": "object",
"properties": {
"query": {"type": "string"},
"user_id": {"type": "string"},
"limit": {"type": "integer", "default": 5}
},
"required": ["query"]
}
},
```

```
{
"name": "update_memory",
"description": "Update existing memory",
"inputSchema": {
"type": "object",
"properties": {
"memory_id": {"type": "string"},
"new_content": {"type": "string"}
},
"required": ["memory_id", "new_content"]
}
},
{
"name": "list_memories",
"description": "List all memories for a user",
"inputSchema": {
"type": "object",
"properties": {
"user_id": {"type": "string"},
"memory_type": {"enum": ["user", "session", "agent"]},
"limit": {"type": "integer", "default": 10}
},
"required": ["user_id"]
}
}
]
```

class MemoryMCPServer:
def **init**(self, memory_client):
self.server = Server("memory-operations")
self.memory_client = memory_client

```
async def handle_add_memory(self, user_id: str, content: str,
                            memory_type: str, metadata: dict):
    """Add memory via MCP"""
    result = await self.memory_client.add_memory(
        user_id=user_id,
        content=content,
        memory_type=memory_type,
        metadata=metadata
    )
    return {"status": "success", "memory_id": result.get("id")}

async def handle_search_memory(self, query: str, user_id: str = None,
                               limit: int = 5):
```

```
        """Search memories via MCP"""
        results = await self.memory_client.search_memory(
            query=query,
            user_id=user_id,
            limit=limit
        )
        return {"status": "success", "results": results}

    async def register_tools(self):
        """Register all memory tools"""
        for tool in MEMORY_TOOLS:
            self.server.add_tool(tool["name"], tool)
```

Export for deployment

```
async def run_mcp_server():
async with stdio_server(MemoryMCPServer) as server:
await server.serve()
```

Part 3: Control Dashboard (React/Next.js)

3.1 Dashboard Architecture

Key Features:

- Real-time system monitoring (Weaviate, PostgreSQL, Firecrawl status)
- Bulk document upload and processing
- Web crawl job management and scheduling
- Memory visualization and search interface
- Multi-tenant management (admin only)
- API key management per tenant
- Rate limiting and quota management
- Audit log viewer

3.2 Next.js Project Setup

Create dashboard project

```
mkdir /data/dashboard
cd /data/dashboard

npx create-next-app@latest --typescript --tailwind --eslint
```

Install dependencies

npm install
axios swr react-query
zustand jotai
recharts
react-data-table-component
date-fns
js-cookie

Install backend integration

npm install @supabase/supabase-js

3.3 Core Dashboard Components

API Client (lib/api.ts):

```
import axios from 'axios';

const API_BASE = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';

export const apiClient = axios.create({
baseURL: API_BASE,
headers: {
'Authorization': Bearer ${process.env.NEXT_PUBLIC_API_KEY}
}
});

// Weaviate endpoints
export const weaviateAPI = {
getCollections: () => apiClient.get('/v1/schema'),
searchDocuments: (query: string, tenantId: string) =>
apiClient.post('/v1/graphql', {
query: { Get { Documents(where: {path: ["tenant_id"], operator: Equal, valueString:
"${tenantId}"}) { title content source_url } } }
}),
addDocument: (doc: any) => apiClient.post('/documents', doc)
};

// Firecrawl endpoints
export const firecrawlAPI = {
scrapeUrl: (url: string) =>
apiClient.post('/crawl/scrape', { url }),
crawlWebsite: (url: string, maxPages: number = 50) =>
apiClient.post('/crawl/website', { url, max_pages: maxPages }),
getJobStatus: (jobId: string) =>
apiClient.get(/crawl/status/${jobId})
};

// mem0 endpoints
export const memoryAPI = {
```

```
addMemory: (userId: string, content: string, type: string) =>
apiClient.post('/memory/add', { user_id: userId, content, memory_type: type }),
searchMemory: (query: string, userId?: string) =>
apiClient.post('/memory/search', { query, user_id: userId }),
listMemories: (userId: string, type?: string) =>
apiClient.get(/memory/list/${userId}, { params: { type } })
};

// System status
export const systemAPI = {
getHealth: () => apiClient.get('/health'),
getStats: () => apiClient.get('/stats')
};
```

Dashboard Layout Component:

```
// components/Layout.tsx
import React from 'react';
import Sidebar from './Sidebar';
import Header from './Header';

export default function Layout({ children }: { children: React.ReactNode }) {
return (



{children}

);
}
```

System Status Page:

```
// app/dashboard/status/page.tsx
'use client';
import { useEffect, useState } from 'react';
import { systemAPI } from '@/lib/api';
import StatusCard from '@/components/StatusCard';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip } from 'recharts';

export default function StatusPage() {
const [status, setStatus] = useState(null);
const [stats, setStats] = useState(null);

useEffect(() => {
const interval = setInterval(async () => {
const [healthRes, statsRes] = await Promise.all([
systemAPI.getHealth(),
systemAPI.getStats()
]);
setStatus(healthRes.data);
setStats(statsRes.data);
}, 5000);
```

```
  return () => clearInterval(interval);
```

}, []);

return (

System Status

```
    <div className="grid grid-cols-4 gap-4">
      <StatusCard
        title="Weaviate"
        status={status?.weaviate_ready}
        metrics={stats?.weaviate_objects}
      />
      <StatusCard
        title="PostgreSQL"
        status={status?.postgres_ready}
        metrics={stats?.documents_count}
      />
      <StatusCard
        title="Firecrawl"
        status={status?.firecrawl_ready}
        metrics={stats?.active_crawls}
      />
      <StatusCard
        title="mem0"
        status={status?.mem0_ready}
        metrics={stats?.total_memories}
      />
    </div>

    <div className="bg-white p-6 rounded-lg shadow">
      <h2 className="text-xl font-semibold mb-4">Memory Growth</h2>
      <LineChart width={800} height={300} data={stats?.memory_timeline || []}>
        <CartesianGrid />
        <XAxis dataKey="timestamp" />
        <YAxis />
```

```
        <Tooltip />
        <Line type="monotone" dataKey="count" stroke="#3b82f6" />
      </LineChart>
    </div>
  </div>
```

```
);
}
```

Crawl Manager Component:

```
// app/dashboard/crawl/page.tsx
'use client';
import { useState } from 'react';
import { firecrawlAPI } from '@/lib/api';
import CrawlJobTable from '@/components/CrawlJobTable';
import useSWR from 'swr';

export default function CrawlPage() {
const [url, setUrl] = useState('');
const [maxPages, setMaxPages] = useState(50);
const [loading, setLoading] = useState(false);

const { data: jobs, mutate } = useSWR('/crawl/jobs',
() => firecrawlAPI.getJobStatus('').catch(() => [])
);

const handleCrawl = async (e: React.FormEvent) => {
e.preventDefault();
setLoading(true);
```

```
  try {
    const result = await firecrawlAPI.crawlWebsite(url, maxPages);
    console.log('Crawl started:', result.data.job_id);
    setUrl('');
    mutate();
  } catch (error) {
    console.error('Crawl failed:', error);
  } finally {
    setLoading(false);
  }
```

```
};

return (
<div className="space-y-6">
```

Web Crawler

```
<form onSubmit={handleCrawl} className="bg-white p-6 rounded-lg shadow
  <div className="space-y-4">
    <div>
      <label className="block text-sm font-medium mb-2">Website URL</label>
      <input
        type="url"
        value={url}
        onChange={(e) => setUrl(e.target.value)}
        placeholder="https://example.com"
        required
        className="w-full px-4 py-2 border rounded-lg focus:ring-2 focus:ring-bl
      />
    </div>

    <div>
      <label className="block text-sm font-medium mb-2">Max Pages</label>
      <input
        type="number"
        value={maxPages}
        onChange={(e) => setMaxPages(Number(e.target.value))}
        min="1"
        max="100"
        className="w-full px-4 py-2 border rounded-lg"
      />
    </div>

    <button
      type="submit"
      disabled={loading}
      className="w-full px-6 py-2 bg-blue-600 text-white rounded-lg hover:bg-b
    >
      {loading ? 'Crawling...' : 'Start Crawl'}
    </button>
  </div>
</form>
```

```
    <div className="bg-white p-6 rounded-lg shadow">
      <h2 className="text-xl font-semibold mb-4">Recent Jobs</h2>
      <CrawlJobTable jobs={jobs || []} />
    </div>
  </div>
```

);
}

Memory Search Component:
// app/dashboard/memory/page.tsx
'use client';
import { useState } from 'react';
import { memoryAPI } from '@/lib/api';

export default function MemoryPage() {
const [query, setQuery] = useState('');
const [results, setResults] = useState([]);
const [loading, setLoading] = useState(false);

const handleSearch = async (e: React.FormEvent) => {
e.preventDefault();
setLoading(true);

```
  try {
    const res = await memoryAPI.searchMemory(query);
    setResults(res.data.results || []);
  } catch (error) {
    console.error('Search failed:', error);
  } finally {
    setLoading(false);
  }
```

};

return (

Memory Search

```
    <form onSubmit={handleSearch} className="bg-white p-6 rounded-lg shadov
      <div className="flex gap-2">
```

```
        <input
          type="text"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Search memories..."
          className="flex-1 px-4 py-2 border rounded-lg focus:ring-2 focus:ring-blu
        />
        <button
          type="submit"
          disabled={loading}
          className="px-6 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700
        >
          Search
        </button>
      </div>
    </form>

    <div className="space-y-4">
      {results.map((result: any, i: number) => (
        <div key={i} className="bg-white p-4 rounded-lg shadow border-l-4 border
          <h3 className="font-semibold text-lg">{result.title}</h3>
          <p className="text-gray-600 mt-2">{result.content}</p>
          <div className="mt-3 flex justify-between text-sm text-gray-500">
            <span>Type: {result.memory_type}</span>
            <span>Score: {(result.relevance_score * 100).toFixed(0)}%</span>
          </div>
        </div>
      ))}
    </div>
  </div>
```

);
}

Part 4: Advanced Features & Integration

4.1 Multi-Tenant Implementation

Tenant Isolation Middleware:

/data/scripts/tenant_middleware.py

from fastapi import Request, HTTPException, Depends
from typing import Optional
import jwt

async def get_tenant_from_request(request: Request) -> str:
"""Extract and validate tenant from JWT token"""
auth_header = request.headers.get("Authorization")
if not auth_header:
raise HTTPException(status_code=401, detail="No authorization header")

```
try:
    token = auth_header.split(" ")[1]
    decoded = jwt.decode(token, "your-secret-key", algorithms=["HS256"])
    return decoded["tenant_id"]
except Exception as e:
    raise HTTPException(status_code=401, detail="Invalid token")
```

async def isolate_tenant_query(tenant_id: str, collection_name: str = "Documents"):
"""Ensure Weaviate queries are tenant-isolated"""
return {
"path": ["tenant_id"],
"operator": "Equal",
"valueString": tenant_id
}

Usage in FastAPI route

@app.post("/search")
async def search_documents(
query: str,
tenant_id: str = Depends(get_tenant_from_request),
client = Depends(get_weaviate_client)
):
where_filter = await isolate_tenant_query(tenant_id)
# Execute search with tenant isolation
pass

4.2 Weaviate + mem0 Integration

Unified Memory Search:

/data/scripts/memory_search.py

from weaviate import Client
from mem0 import Memory

class UnifiedMemorySearch:
def **init**(self, weaviate_client, mem0_instance):
self.weaviate = weaviate_client
self.mem0 = mem0_instance

```
async def hybrid_search(self, query: str, tenant_id: str, user_id: str):
    """Search both Weaviate (semantic) and mem0 (personalized)"""

    # Semantic search in Weaviate
    semantic_results = self.weaviate.query.get(
        "Documents",
        ["title", "content", "source_url"]
    ).with_where({
        "operator": "And",
        "operands": [
            {"path": ["tenant_id"], "operator": "Equal", "valueString": tenant_id},
            {"path": ["vectorized"], "operator": "Equal", "valueBoolean": True}
        ]
    }).with_near_text({"concepts": [query]}).with_limit(5).do()

    # Personalized memory search in mem0
    mem0_results = self.mem0.search(
        query=query,
        user_id=f"{tenant_id}:{user_id}",
        limit=5
    )

    # Combine and rank results
    combined = {
        "semantic": semantic_results.get("data", {}).get("Get", {}).get("Documents", []
        "personalized": mem0_results
    }
```

```
    return combined
```

4.3 Automated Crawling & Indexing Pipeline

n8n Workflow (Optional but Recommended):

- Create n8n workflow file: `/data/configs/crawl-workflow.json`
- Triggered on schedule (e.g., daily at 2 AM)
- Fetch URLs from PostgreSQL
- Call Firecrawl API
- Transform markdown to structured format
- Index into Weaviate
- Log results to PostgreSQL

Alternative Python Scheduler:

/data/scripts/scheduled_crawler.py

from apscheduler.schedulers.background import BackgroundScheduler
from firecrawl_client import FirecrawlClient
import psycopg2
import weaviate

scheduler = BackgroundScheduler()
fc = FirecrawlClient()

async def scheduled_crawl_job():
"""Run scheduled crawls"""
conn = psycopg2.connect("dbname=memoryserver user=memoryserver")
cursor = conn.cursor()

```
# Get pending URLs
cursor.execute("""
    SELECT id, url FROM crawl_jobs
    WHERE status = 'pending' LIMIT 10
""")

for job_id, url in cursor.fetchall():
    try:
        # Scrape
        result = fc.scrape_page(url)
        content = result.get("data", {}).get("markdown", "")

        # Index to Weaviate
```

```
        doc = {
            "title": result.get("title", url),
            "content": content,
            "source_url": url,
            "tenant_id": "global"
        }

        # Insert to Weaviate
        # Then update PostgreSQL
        cursor.execute("""
            UPDATE crawl_jobs
            SET status = 'completed', result = %s
            WHERE id = %s
        """, (json.dumps(result), job_id))

    except Exception as e:
        cursor.execute("""
            UPDATE crawl_jobs
            SET status = 'failed', result = %s
            WHERE id = %s
        """, (json.dumps({"error": str(e)}), job_id))

    conn.commit()

cursor.close()
conn.close()
```

Schedule runs every 6 hours

scheduler.add_job(scheduled_crawl_job, 'interval', hours=6)
scheduler.start()

4.4 Advanced Document Conversion

File Processing Pipeline (/data/scripts/doc_processor.py):
from pathlib import Path
import subprocess
import PyPDF2
import markdown
from docx import Document as DocxDocument

class DocumentProcessor:
@staticmethod
def pdf_to_markdown(pdf_path: str) -> str:
"""Convert PDF to markdown"""
with open(pdf_path, 'rb') as f:
reader = PyPDF2.PdfReader(f)
text = ""
for page in reader.pages:
text += page.extract_text()
return text

```
@staticmethod
def docx_to_markdown(docx_path: str) -> str:
    """Convert DOCX to markdown"""
    doc = DocxDocument(docx_path)
    md = ""
    for para in doc.paragraphs:
        md += f"{para.text}\n"
    for table in doc.tables:
        for row in table.rows:
            md += " | ".join(cell.text for cell in row.cells) + "\n"
    return md

@staticmethod
def csv_to_markdown(csv_path: str) -> str:
    """Convert CSV to markdown table"""
    import pandas as pd
    df = pd.read_csv(csv_path)
    return df.to_markdown(index=False)

@staticmethod
def html_to_markdown(html_content: str) -> str:
    """Convert HTML to markdown"""
    from html2text import html2text
    return html2text(html_content)

@classmethod
def process_file(cls, file_path: str) -> str:
    """Auto-detect and convert any document type"""
    ext = Path(file_path).suffix.lower()
```

```
    if ext == '.pdf':
        return cls.pdf_to_markdown(file_path)
    elif ext == '.docx':
        return cls.docx_to_markdown(file_path)
    elif ext == '.csv':
        return cls.csv_to_markdown(file_path)
    elif ext == '.html':
        with open(file_path) as f:
            return cls.html_to_markdown(f.read())
    elif ext in ['.txt', '.md']:
        with open(file_path) as f:
            return f.read()
    else:
        raise ValueError(f"Unsupported file type: {ext}")
```

Part 5: Deployment & Operations

5.1 Complete Docker Compose Stack

Master Compose File (/data/docker-compose.yml):
version: '3.8'

services:

Vector Database

weaviate:
image: semitechnologies/weaviate:latest
ports:
- "8080:8080"
- "50051:50051"
environment:
AUTHENTICATION_APIKEY_ENABLED: 'true'
AUTHENTICATION_APIKEY_ALLOWED_KEYS: 'weaviate-key-prod'
MULTI_TENANCY_ENABLED: 'true'
PERSISTENCE_DATA_PATH: '/var/lib/weaviate'
volumes:
- weaviate_data:/var/lib/weaviate
restart: unless-stopped

Document & Log Store

postgres:
image: postgres:16-alpine
environment:
POSTGRES_USER: memoryserver
POSTGRES_PASSWORD: ${DB_PASSWORD}
ports:
- "5432:5432"
volumes:
- postgres_data:/var/lib/postgresql/data
- /data/configs/init.sql:/docker-entrypoint-initdb.d/init.sql
restart: unless-stopped

Cache & Queue

redis:
image: redis:7-alpine
ports:
- "6379:6379"
volumes:
- redis_data:/data
restart: unless-stopped

Web Crawler

firecrawl:
image: node:18-alpine
working_dir: /app
ports:
- "3002:3002"
environment:
REDIS_URL: redis://redis:6379
PORT: 3002
depends_on:
- redis
restart: unless-stopped

Memory Layer

mem0:
build:
context: /data/configs
dockerfile: Dockerfile.mem0
ports:
- "8001:8000"
environment:

WEAVIATE_URL: http://weaviate:8080
POSTGRES_URL:
postgresql://memoryserver:${DB_PASSWORD}@postgres:5432/memoryserver
OPENAI_API_KEY: ${OPENAI_API_KEY}
depends_on:
- weaviate
- postgres
restart: unless-stopped

Custom MCP Server

mcp_server:
build:
context: /data/scripts
dockerfile: Dockerfile.mcp
ports:
- "8002:8000"
environment:
MEM0_URL: http://mem0:8000
WEAVIATE_URL: http://weaviate:8080
depends_on:
- mem0
- weaviate
restart: unless-stopped

Control Dashboard

dashboard:
build:
context: /data/dashboard
ports:
- "3000:3000"
environment:
NEXT_PUBLIC_API_URL: http://localhost:8080
NEXT_PUBLIC_API_KEY: ${API_KEY}
depends_on:
- weaviate
- postgres
- firecrawl
- mem0
restart: unless-stopped

Reverse Proxy

nginx:
image: nginx:latest
ports:
- "80:80"

- "443:443"
volumes:
- /data/configs/nginx.conf:/etc/nginx/nginx.conf
depends_on:
- dashboard
- weaviate
- firecrawl
restart: unless-stopped

volumes:
weaviate_data:
postgres_data:
redis_data:

Launch Everything:
cd /data
export DB_PASSWORD="secure-password-here"
export OPENAI_API_KEY="sk-..."
export API_KEY="your-dashboard-key"

docker compose up -d

Verify all services

docker compose ps

5.2 Reverse Proxy Configuration

Nginx Config (`/data/configs/nginx.conf`):
upstream weaviate_backend {
server weaviate:8080;
}

upstream firecrawl_backend {
server firecrawl:3002;
}

upstream dashboard_backend {
server dashboard:3000;
}

upstream mem0_backend {
server mem0:8000;
}

server {
listen 80;
server_name memory.yourdomain.com;

```
# Dashboard
location / {
```

```
    proxy_pass http://dashboard_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }

  # Weaviate API
  location /api/weaviate {
    rewrite ^/api/weaviate(.*)$ $1 break;
    proxy_pass http://weaviate_backend;
    auth_request /auth;
  }

  # Firecrawl API
  location /api/crawl {
    rewrite ^/api/crawl(.*)$ $1 break;
    proxy_pass http://firecrawl_backend;
    auth_request /auth;
  }

  # mem0 API
  location /api/memory {
    rewrite ^/api/memory(.*)$ $1 break;
    proxy_pass http://mem0_backend;
    auth_request /auth;
  }

  # Auth endpoint
  location /auth {
    proxy_pass http://dashboard_backend/api/auth;
    proxy_pass_request_body off;
    proxy_set_header Content-Length "";
  }

}
```

5.3 Backup & Recovery Strategy

Daily Backup Script (`/data/scripts/backup.sh`):

```
#!/bin/bash

BACKUP_DIR="/data/backups/$(date +%Y%m%d)"
mkdir -p $BACKUP_DIR

# Backup PostgreSQL

docker exec postgres_memory pg_dump -U memoryserver memoryserver |
gzip > $BACKUP_DIR/postgres_backup.sql.gz

# Backup Weaviate data

tar -czf $BACKUP_DIR/weaviate_backup.tar.gz
/data/volumes/weaviate

# Backup Redis

docker exec firecrawl_redis redis-cli BGSAVE
cp /data/volumes/redis/dump.rdb $BACKUP_DIR/redis_backup.rdb

# Upload to cloud storage (optional)

# aws s3 cp
$BACKUP_DIRs3://my-backups/
# (date +%Y%m%d) --recursive

echo "✓ Backup completed: $BACKUP_DIR"

# Keep only last 30 days

find /data/backups -type d -mtime +30 -exec rm -rf {} ;
```

Add to crontab:

```
crontab -e
```

Add: 0 2 * * * /data/scripts/backup.sh

Part 6: MCP Integration & Usage Examples

6.1 Using mem0 MCP with Claude

/data/examples/claude_with_memory.py

from anthropic import Anthropic
import json
from mem0 import Memory

client = Anthropic()

Initialize multi-tenant mem0

mem0_config = {
"vector_store": {
"provider": "weaviate",
"config": {
"cluster_url": "http://localhost:8080",
"auth_client_secret": "weaviate-key-prod"
}
},
"llm": {
"provider": "openai",
"config": {"model": "gpt-4"}
}
}

memory = Memory.from_config(mem0_config)

class MemorizedAgent:
def **init**(self, user_id: str, tenant_id: str = "default"):
self.user_id = user_id
self.tenant_id = tenant_id
self.conversation_history = []
self.memory = memory

```
def get_relevant_memories(self, query: str, limit: int = 3) -> list:
    """Retrieve relevant memories before responding"""
    memories = self.memory.search(
        query=query,
        user_id=f"{self.tenant_id}:{self.user_id}",
        limit=limit
```

```
    )
    return memories

def chat(self, user_message: str) -> str:
    """Process message with memory context"""

    # Get relevant memories
    relevant_memories = self.get_relevant_memories(user_message)

    # Build memory context
    memory_context = ""
    if relevant_memories:
        memory_context = "\n\nRELEVANT MEMORIES:\n"
        for mem in relevant_memories:
            memory_context += f"- {mem['content']}\n"

    # Add to conversation
    self.conversation_history.append({
        "role": "user",
        "content": user_message + memory_context
    })

    # Get response from Claude
    response = client.messages.create(
        model="claude-3-5-sonnet-20241022",
        max_tokens=2048,
        system=f"You are a helpful assistant with persistent memory. Use the relev
        messages=self.conversation_history
    )

    assistant_message = response.content[0].text
    self.conversation_history.append({
        "role": "assistant",
        "content": assistant_message
    })

    # Store interaction in memory
    self.memory.add(
```

```
        messages=[{
            "role": "user",
            "content": user_message
        }, {
            "role": "assistant",
            "content": assistant_message
        }],
        user_id=f"{self.tenant_id}:{self.user_id}",
        metadata={"type": "conversation"}
    )

    return assistant_message
```

Usage

agent = MemorizedAgent(user_id="user123", tenant_id="tenant1")
response = agent.chat("Tell me about my recent project status")
print(response)

6.2 Building Knowledge Graph

/data/examples/knowledge_graph.py

from typing import List, Dict
import weaviate

class KnowledgeGraphBuilder:
def **init**(self, weaviate_client):
self.client = weaviate_client

```
def create_entity_collection(self):
    """Create entity and relationship collections"""
    self.client.collections.create(
        name="Entities",
        properties=[
            {"name": "name", "dataType": ["string"]},
            {"name": "type", "dataType": ["string"]},
            {"name": "description", "dataType": ["text"]},
            {"name": "tenant_id", "dataType": ["string"]}
        ]
```

```
    )

    self.client.collections.create(
        name="Relationships",
        properties=[
            {"name": "source_entity", "dataType": ["string"]},
            {"name": "relationship_type", "dataType": ["string"]},
            {"name": "target_entity", "dataType": ["string"]},
            {"name": "weight", "dataType": ["number"]},
            {"name": "tenant_id", "dataType": ["string"]}
        ]
    )

def add_entity(self, name: str, entity_type: str, description: str, tenant_id: str) -> s
    """Add entity to knowledge graph"""
    entity_id = f"{tenant_id}:{entity_type}:{name}".lower().replace(" ", "_")

    self.client.collections.get("Entities").data.create(
        properties={
            "name": name,
            "type": entity_type,
            "description": description,
            "tenant_id": tenant_id
        },
        uuid=entity_id
    )

    return entity_id

def add_relationship(self, source_id: str, relationship: str,
                     target_id: str, weight: float = 1.0, tenant_id: str = "default"):
    """Add relationship between entities"""
    self.client.collections.get("Relationships").data.create(
        properties={
            "source_entity": source_id,
            "relationship_type": relationship,
            "target_entity": target_id,
            "weight": weight,
```

```
            "tenant_id": tenant_id
        }
    )

def find_related_entities(self, entity_id: str, depth: int = 2) -> Dict:
    """Find all entities related within N hops"""
    related = {entity_id: []}
    visited = {entity_id}
    queue = [(entity_id, 0)]

    while queue:
        current_id, current_depth = queue.pop(0)

        if current_depth >= depth:
            continue

        # Query relationships
        rels = self.client.collections.get("Relationships").query\
            .where({"path": ["source_entity"], "operator": "Equal", "valueString": curr
            .fetch_objects()

        for rel in rels.objects:
            target_id = rel.properties["target_entity"]
            if target_id not in visited:
                visited.add(target_id)
                related[entity_id].append({
                    "entity_id": target_id,
                    "relationship": rel.properties["relationship_type"],
                    "weight": rel.properties["weight"]
                })
                queue.append((target_id, current_depth + 1))

    return related
```

Part 7: Performance Tuning & Monitoring

7.1 Weaviate Performance Optimization

Configure HNSW indexing for faster similarity search

```
{
"vectorIndexConfig": {
"skip": False,
"cleanupIntervalSeconds": 300,
"maxConnections": 64,
"efConstruction": 128,
"ef": 128,
"dynamicEfMin": 100,
"dynamicEfMax": 500,
"dynamicEfFactor": 8,
"vectorCacheMaxObjects": 1000000,
"flatSearchCutoff": 40000,
"distance": "cosine"
}
}
```

7.2 PostgreSQL Optimization

```
-- Connection pooling
ALTER SYSTEM SET max_connections = 200;
ALTER SYSTEM SET shared_buffers = '16GB';
ALTER SYSTEM SET effective_cache_size = '48GB';
ALTER SYSTEM SET work_mem = '40MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET random_page_cost = 1.1;
ALTER SYSTEM SET effective_io_concurrency = 200;

-- Index optimization
REINDEX INDEX CONCURRENTLY idx_documents_tenant;
VACUUM ANALYZE;
```

7.3 Monitoring & Alerting

Prometheus Metrics (`/data/configs/prometheus.yml`):

```
global:
scrape_interval: 15s

scrape_configs:

- job_name: 'weaviate'
  static_configs:
    - targets: ['localhost:8080']
```

- job_name: 'postgres'
  static_configs:
    - targets: ['localhost:9187']
- job_name: 'firecrawl'
  static_configs:
    - targets: ['localhost:3002']

References

[1] Weaviate Documentation. (2024). Multi-tenancy in Weaviate. https://weaviate.io/developers/weaviate/manage-data/multi-tenancy

[2] Mem0 Team. (2024). mem0 - Universal Memory Layer for AI Agents. https://github.com/mem0ai/mem0

[3] Firecrawl. (2024). Self-hosting Firecrawl. https://docs.firecrawl.dev/self-hosting

[4] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

[5] Weaviate Team. (2024). Weaviate Vector Database. https://weaviate.io

[6] Redis Foundation. (2024). Redis Documentation. https://redis.io/docs

[7] N8N. (2024). n8n Workflow Automation. https://n8n.io/docs

[8] Next.js Documentation. (2024). Next.js React Framework. https://nextjs.org/docs

[9] Canonical. (2024). Ubuntu AI/ML Projects. https://ubuntu.com/blog/ubuntu-ai-ml-projects

[10] Foster Fletcher. (2025). AI Memory Infrastructure: Mem0 vs. OpenMemory. https://fosterfletcher.com/ai-memory-infrastructure

---

Appendix: Quick Start Commands

Full system deployment

```
cd /data
export DB_PASSWORD="your-secure-password"
export OPENAI_API_KEY="sk-your-key"
docker compose up -d
```

Check system health

```
curl http://localhost:8080/.well-known/ready # Weaviate
curl http://localhost:3002/health # Firecrawl
psql -h localhost -U memoryserver -d memoryserver # PostgreSQL
```

Access services

Dashboard: http://localhost:3000
Weaviate: http://localhost:8080
Firecrawl: http://localhost:3002
PostgreSQL: localhost:5432
Redis: localhost:6379

View logs

```
docker compose logs -f weaviate
docker compose logs -f postgres
docker compose logs -f firecrawl
```